04 - Functions

A loop saved us from writing the same three lines out three hundred times. This notebook is about the other kind of repetition: the same *calculation* appearing in several different places in a program, and drifting apart as we edit them one at a time.

The answer is a **function** - a piece of code with a name, which we can use again without writing it out again. This notebook covers:

- Defining a function: `def`, parameters and `return`
- `return` versus `print`
- Arguments, and giving them default values
- Scope: what a function can and cannot see
- Docstrings, and `help()` on your own code
- Building a whole program out of functions
- Errors and exceptions, and how to read a traceback
- `try` / `except`, and catching the right thing

Why we need functions

Here is a program that reports three temperature readings in Fahrenheit. But the third print statement is wrong: it has `5 / 9` where the other two have `9 / 5`.

In [ ]:
```
oslo = 4.2
bergen = 7.8
tromso = -3.1

print(f"Oslo:    {9 / 5 * oslo + 32:.1f} F")
print(f"Bergen:  {9 / 5 * bergen + 32:.1f} F")
print(f"Tromso:  {5 / 9 * tromso + 32:.1f} F") # typo
```

The formula is written out three times, so it can be wrong in three different ways. Written **once**, under a name, it can only be wrong in one - and once you have fixed it, it is fixed everywhere.

We have been using functions since the first week without writing any:

In [ ]:
```
print(len("Bergen"))
print(type(4.2))
print(round(3.14159, 2))
```

Each of those has four things worth naming:

Loading [MathJax]/extensions/Safe.js

`len`	
Name	`len`
Task	measure how long a sequence is
Input	a sequence
Output	a whole number

This notebook is about writing our own functions.

Defining a function

A function definition has a **header** and a **body**.

```
def <name>(<parameters>):
    <statements>
    return <value>
```

The header starts with `def`, gives the function a name, lists its **parameters** in brackets, and ends with a **colon**. The body is **indented** underneath, exactly as it was for `if` and for `for`. The `return` statement says what the function hands back.

In [ ]:
```
def to_fahrenheit(celsius):
    fahrenheit = 9 / 5 * celsius + 32
    return fahrenheit
```

Running that cell produced no output, and that is correct. A definition does not *do* anything - it teaches Python a new word. Nothing happens until we **call** the function.

In [ ]:
```
print(to_fahrenheit(4.2))
print(to_fahrenheit(100))
print(to_fahrenheit(-40))
```

One definition, three uses, one place to fix if the formula is wrong.

A call hands back a value like any other expression, so we can store it, or use it inside something larger.

In [ ]:
```
oslo_f = to_fahrenheit(4.2)

print(f"Oslo is {oslo_f:.1f} F")
print(f"The difference is {to_fahrenheit(7.8) - to_fahrenheit(4.2):.1f} degr
```

Loading [MathJax]/extensions/Safe.js

Applying a function to a whole collection

Our readings so far came one at a time. More often they arrive together in a list, and the obvious thing to try is handing the function the whole list.

In [ ]:
```
# TypeError
temperatures_c = [4.2, 18.5, 11.0, 22.3]

to_fahrenheit(temperatures_c)
```

Read the last line of the error: Python cannot multiply a *sequence* by a decimal number. `to_fahrenheit` was written for **one** number, and a list is not a number. Plain Python lists do not do arithmetic on every item at once - you will meet tools later in the course that do.

A function written for one value is applied to a whole list with the comprehension we already have, which calls it once per item.

In [ ]:
```
temperatures_c = [4.2, 18.5, 11.0, 22.3]

temperatures_f = [to_fahrenheit(t) for t in temperatures_c]

print([round(t, 1) for t in temperatures_f])
```

Two things about the name. Functions follow the same **snake case** convention as variables - lowercase words joined by underscores. And a function name should usually be a **verb or a phrase describing what comes back**, because a function *does* something, whereas a variable *is* something: `to_fahrenheit`, `count_words`, `is_valid`.

> 📝 **Note:** A function must be defined **before** it is called. In a notebook that means the `def` cell has to have been run. If you restart the kernel and jump straight to a call, you get a `NameError` telling you the name is not defined - which is the same message you get for a misspelled variable, and means the same thing.

Your turn

Conversion the other way round follows the formula `C = 5 / 9 * (F - 32)`.

Write a function called `to_celsius` that takes a temperature in Fahrenheit and **returns** it in Celsius, then check it on these three values:

```
32      212      -40
```

Loading [MathJax]/extensions/Safe.js

> The last one is a useful test: -40 is the temperature at which the two scales agree.

In [ ]:
```
```

`return` is not `print`

This is one of the most common misunderstanding about functions.

Here is the same conversion written to display its answer instead of returning it.

In [ ]:
```
def show_fahrenheit(celsius):
    fahrenheit = 9 / 5 * celsius + 32
    print(f"{celsius} C is {fahrenheit:.1f} F")


show_fahrenheit(4.2)
```

It looks like it worked. It did not - or rather, it did exactly one thing and gave us nothing. Watch what happens when we try to use the answer.

In [ ]:
```
# TypeError
average = (show_fahrenheit(4.2) + show_fahrenheit(7.8)) / 2
```

The two temperatures were displayed, and then the program failed, because `show_fahrenheit` has no value to give back. `print` puts characters on the screen; it does not hand anything to the code that called it.

A function that prints is a dead end. A function that returns can be used again. So the rule is: compute and return in the function, and display in the code that calls it.

In [ ]:
```
print(f"The average is {(to_fahrenheit(4.2) + to_fahrenheit(7.8)) / 2:.1f} F
```

A function with no `return` gives back `None`

We have met `None` before: Python's way of saying *there is no value here*.

In [ ]:
```
result = show_fahrenheit(4.2)

print(result)
print(type(result))
```

So a function without a `return` is not a function that returns nothing - it is a function that returns `None`, which is a value, and which will happily travel through your program until it meets something that cannot cope with it.

Loading [MathJax]/extensions/Safe.js

`return` leaves the function immediately

Nothing after a `return` in the same block ever runs.

In [ ]:
```
def double(number):
    return number * 2
    print("This line never runs.")


print(double(21))
```

That is not a limitation - it is how a function with several answers is written. Each branch returns as soon as it knows the answer, and there is no need for an `else`.

In [ ]:
```
def describe(number):
    if number > 0:
        return "positive"
    if number < 0:
        return "negative"
    return "zero"


print(describe(4.2))
print(describe(-1))
print(describe(0))
```

`return` inside a loop leaves the **function**, not just the loop

We know that `break` leaves the innermost loop and nothing more. `return` is stronger: it abandons the loop *and* the function, in one go. In a search that is usually exactly what you want.

In [ ]:
```
def first_negative(values):
    for value in values:
        if value < 0:
            return value

    return None


print(first_negative([4, 9, -2, 7, -5]))
print(first_negative([4, 9, 7]))
```

Notice the last line of that function. Without it, a list containing no negative number would fall off the end of the function body and return `None` **by accident** rather than on purpose. Both versions behave the same way; only one of them says so.

Returning a condition

Loading [MathJax]/extensions/Safe.js

A function that answers a yes-or-no question does not need an `if` at all. A comparison is already `True` or `False`, so it can be returned directly.

In [ ]:
```
def is_valid_scale(scale):
    if scale in ("C", "F"):
        return True
    else:
        return False


def is_valid_scale_short(scale):
    return scale in ("C", "F")


print(is_valid_scale("C"), is_valid_scale_short("C"))
print(is_valid_scale("K"), is_valid_scale_short("K"))
```

Four lines become one, and the short version is easier to read aloud: *return whether the scale is C or F*. Whenever you catch yourself writing `if <condition>: return True else: return False`, return the condition.

Your turn

This function should return `"pass"` or `"fail"`, but `print(grade(45))` displays something else:

```
def grade(score):
    if score >= 60:
        return "pass"
```

Fix the function, then check it on `45`, `60` and `80`.

In [ ]:
```

```

Arguments

The names in the function header are its **parameters**. The values we hand over in the call are its **arguments**. There is more than one way to hand them over.

In [ ]:
```
def convert(temperature, scale):
    if scale == "C":
        return 9 / 5 * temperature + 32
    if scale == "F":
        return 5 / 9 * (temperature - 32)

    return None
```

Loading [MathJax]/extensions/Safe.js

By **position**: the first argument goes to the first parameter, the second to the second. Order is the whole story.

In [ ]:
```
print(convert(100, "C"))
print(convert(100, "F"))
```

By **name**: write `parameter=value` and the order stops mattering. It is longer, and it is worth it whenever a call would otherwise be a row of anonymous values.

In [ ]:
```
print(convert(temperature=100, scale="C"))
print(convert(scale="C", temperature=100))
```

The two styles can be mixed, but positional arguments must come first - Python cannot work out what an unnamed value refers to once the order has been abandoned.

A function must be called with all of its parameters filled in.

In [ ]:
```
# TypeError
convert(100)
```

Read that message: it names the function, says how many arguments are missing, and names them. Error messages in Python are usually this good, and reading them is faster than guessing.

Default values

A parameter can be given a **default** in the header. That makes it optional: supply it and yours is used, leave it out and the default is used.

In [ ]:
```
def convert(temperature, scale="C"):
    if scale == "C":
        return 9 / 5 * temperature + 32
    if scale == "F":
        return 5 / 9 * (temperature - 32)

    return None


print(convert(100))
print(convert(100, "F"))
print(convert(100, scale="F"))
```

Defaults are how a function can be simple in the common case and still flexible in the rare one. `print` works this way. We used `print(value, end=" ")` to keep a row of a nested loop on one line: that works because `end` is a parameter with a default of `"\n"`, so `print(x)` ends the line and `print(x, end=" ")` does not.

Loading [MathJax]/extensions/Safe.js

Parameters **with** defaults must come after parameters **without** them. Otherwise there would be calls Python could not interpret, so it refuses the definition outright.

In [ ]:
```
# SyntaxError
def convert(scale="C", temperature):
    return 9 / 5 * temperature + 32
```

Your turn

Write a function called `split_bill` that returns each person's share of a restaurant bill, tip included. It takes the total, the number of people, and a tip rate `tip` that defaults to `0.10`.

Check it with these three calls:

```
split_bill(1200, 4)
split_bill(1200, 4, 0.2)
split_bill(1200, people=4, tip=0)
```

In [ ]:
```
```

Returning more than one value

Separate several values with commas after `return` and the function hands back all of them.

In [ ]:
```
def summarize(values):
    total = sum(values)
    average = total / len(values)
    return total, average


result = summarize([120, 340, 55, 890])

print(result)
print(type(result))
```

What came back is a **tuple** - the immutable cousin of the list we met when looping over `.items()`. And it unpacks the same way: give as many names on the left as there are values, and Python splits them between the names.

In [ ]:
```
total, average = summarize([120, 340, 55, 890])

print(f"Total: {total}, average: {average:.1f}")
```

Loading [MathJax]/extensions/Safe.js

Scope: what a function can see

A variable created inside a function is **local** to it. It comes into existence when the function is called, and disappears when the function returns.

In [ ]:
```
def to_fahrenheit(celsius):
    fahrenheit = 9 / 5 * celsius + 32
    return fahrenheit


print(to_fahrenheit(100))
```

In [ ]:
```
# NameError
print(fahrenheit)
```

That is not an inconvenience - it is the point. Because names inside a function are private to it, two functions can use the same name for different things without colliding - `to_fahrenheit` and `show_fahrenheit` both have a variable called `fahrenheit`, and neither has ever seen the other's - and you can write a function without first checking what every other variable in the program is called.

Global variables, and why to avoid them

A variable defined outside any function is **global**, and a function can see it without being handed it.

In [ ]:
```
vat_rate = 0.25 # global variable


def add_vat(price):
    return price * (1 + vat_rate)


print(add_vat(100))
```

That works, and it is a bad habit. The function's header says it needs one thing, and the function actually needs two - so you cannot tell what it depends on by looking at it, you cannot reuse it in another program without carrying `vat_rate` along.

The repair is to pass it in. A default value keeps the common case short.

In [ ]:
```
def add_vat(price, vat_rate=0.25):
    return price * (1 + vat_rate)


print(add_vat(100))
print(add_vat(100, vat_rate=0.15))
```

> 💡 **Tip:** A good function is a black box: everything it needs comes in through the brackets, and everything it produces comes out through

Loading [MathJax]/extensions/Safe.js

> `return`. If you can read the header and know what the function depends on, you can reuse it anywhere.

A function can change your list

There is one way in which a function is *not* sealed off, and it catches everybody once.

In [ ]:
```
def add_ten(number):
    number = number + 10
    return number


score = 5

print(add_ten(score))
print(score)
```

`score` is untouched, exactly as scope promised. Now the same thing with a list.

In [ ]:
```
def add_item(basket):
    basket.append("Oat milk")
    return basket


my_basket = ["Coffee beans"]

print(add_item(my_basket))
print(my_basket)
```

The original list changed. This is the fact about names and objects from the very first week, seen from a new angle: **a parameter is another name for the same object**, not a copy of it. With a number there is nothing to change in place, so `number = number + 10` just points the local name at a new number. With a list, `.append` reaches into the object that both names refer to.

If a function should leave its input alone, it has to work on a copy.

In [ ]:
```
def with_extra_item(basket):
    new_basket = basket[:]
    new_basket.append("Oat milk")
    return new_basket


my_basket = ["Coffee beans"]

print(with_extra_item(my_basket))
print(my_basket)
```

Loading [MathJax]/extensions/Safe.js

> ⚠️ **Warning:** A function that changes its argument *and* returns it is asking to be misunderstood - the return value makes it look like nothing was touched. Pick one: change the caller's object and return nothing, or leave it alone and return a new one.

Your turn

Starting from this list:

```
readings = [4.2, -3.1, 0.0, -8.5, 12.3]
```

Write a function `without_negatives(values)` that returns a **new** list containing only the values that are zero or above, then display the returned list and `readings` to show that the original is untouched.

In [ ]:

```

```

Documenting a function

A string written on the first line of the body is a **docstring**. It is not a comment - Python keeps it, and hands it back when somebody asks what the function does.

In [ ]:

```
def to_fahrenheit(celsius):
    """Return a Celsius temperature converted to Fahrenheit."""
    return 9 / 5 * celsius + 32


help(to_fahrenheit)
```

Every function in Python has one, which is why `help` works on code you did not write.

In [ ]:

```
from random import randint

help(randint)
```

A single line is enough for a short function, and it should say what the function **returns**, not how it works. When there are several parameters and their meaning is not obvious, the longer form earns its keep:

```
def eff_interest_rate(r, n):
    """
    Return the effective annual interest rate.

    Parameters
    ----------
    r : float
```

Loading [MathJax]/extensions/Safe.js

```
        The nominal annual interest rate, as a decimal.
    n : int
        The number of times per year that interest is compounded.

    Returns
    -------
    float
        The effective annual rate, as a decimal.
    """
    return (1 + r / n) ** n - 1
```

> 💡 **Tip:** In Positron, hovering over a function name shows its docstring, and so does typing `help(name)`. If hovering over your own function shows nothing useful, that is a message about your docstring.

A program built out of functions

Once functions exist, a program stops being a list of statements and becomes a small set of named jobs, plus one function that puts them in order. That last one is conventionally called `main`.

In [ ]:
```
def line_total(quantity, unit_price):
    """Return the total for one receipt line."""
    return quantity * unit_price


def format_line(item, quantity, unit_price):
    """Return one formatted receipt line as a string."""
    return f"{item:<20}{quantity:>3} x {unit_price:>8.2f} = {line_total(quar


def main():
    """Print a receipt for the items below."""
    items = ["Espresso machine", "Coffee beans", "Oat milk"]
    quantities = [1, 2, 3]
    unit_prices = [4999.00, 149.90, 24.50]

    subtotal = 0

    for item, quantity, price in zip(items, quantities, unit_prices):
        print(format_line(item, quantity, price))
        subtotal += line_total(quantity, price)

    print("-" * 45)
    print(f"{'TOTAL':<20}{subtotal:>25,.2f}")
```

In [ ]:
```
main()
```

Loading [MathJax]/extensions/Safe.js

Nothing new is happening - it is the receipt from the last set of exercises. What changed is that each piece has a name and can be tested on its own. `line_total(3, 24.50)` either gives 73.50 or it does not, and you can find that out without printing a receipt.

Notice also that the data lives **inside** `main` rather than at the top of the file. That is the scope lesson applied: there are now no global variables at all, so no function can accidentally depend on one.

Your turn

This program works, but only one piece of it is a function, and its data sits at the top of the file:

```
def to_fahrenheit(celsius):
    return 9 / 5 * celsius + 32


cities = ["Oslo", "Bergen", "Tromso"]
temperatures = [4.2, 7.8, -3.1]

for city, celsius in zip(cities, temperatures):
    print(f"{city:<8}{celsius:>6.1f}
C{to_fahrenheit(celsius):>7.1f} F")
```

Restructure it into a function `format_row(city, celsius)` that **returns** one line as a string, and a `main()` that holds the data and prints the rows, so that no variable is defined outside a function. Then call `main()`.

In [ ]:

Errors and exceptions

Everything from here on is about programs that stop.

A **syntax error** means Python could not read your code at all. Nothing runs - not even the correct lines above the mistake - because the whole cell has to be understood before any of it can be executed. The `SyntaxError` a few sections ago was one of these.

An **exception** is different. The program was perfectly readable, it started running, and then it hit something it could not do: a word that is not a number, a division by zero, a position past the end of a list. Everything before that point already happened. Everything after it did not.

Loading [MathJax]/extensions/Safe.js

```
sionError
print("This line runs.")
```

```
print(10 / 0)

print("This line does not.")
```

The ones you will actually meet

An exception has a **type**, and the type is the useful part - it tells you what kind of thing went wrong, and it is what you will name when you start catching them.

Type	Raised when
`ValueError`	the value is the right kind of thing but the wrong value - `float("abc")`
`TypeError`	the value is the wrong kind of thing altogether - `"3" + 3`
`ZeroDivisionError`	you divided by zero
`IndexError`	a list position that does not exist
`KeyError`	a dictionary key that does not exist
`NameError`	a name that has not been defined - almost always a typo

In [ ]:
```
# ValueError
float("forty two")
```

In [ ]:
```
# TypeError
"3" + 3
```

In [ ]:
```
# KeyError
capitals = {"Norway": "Oslo", "Sweden": "Stockholm"}

capitals["Denmark"]
```

`ValueError` and `TypeError` are worth telling apart. `float("abc")` is a `ValueError` because a string is a perfectly reasonable thing to hand to `float` - this particular string just is not a number. `float([1, 2])` is a `TypeError`, because a list is not the sort of thing `float` converts at all.

None of these is magic. When `float` is handed `"forty two"`, it tries, fails, and **raises** a `ValueError` with a message saying why - using an ordinary statement that you can write yourself.

In [ ]:
```
# ValueError
raise ValueError("that is not a valid temperature")
```

Reading a traceback

Loading [MathJax]/extensions/Safe.js

The report Python prints when an exception is not dealt with is a **traceback**, and once you write functions it stops being one line long. Here is a program in three layers.

In [ ]:
```
def run_report(reading):
    return f"{to_fahrenheit(reading):.1f} F"


# TypeError
print(run_report("12.5"))
```

There is a way to read that, and it is not top to bottom.

1. **Start at the bottom.** The last line is the actual complaint: `TypeError: can't multiply sequence by non-int of type 'float'`. That is what went wrong, and it is the same message we got when we handed `to_fahrenheit` a whole list: a string is a sequence too.
2. **Then go up, and find the topmost frame that is your own code.** Above the message is a stack of frames, innermost last. The innermost is inside `to_fahrenheit`, which is where it *broke*. The one above is `run_report`, and above that is the line you actually ran.
3. **The bug is usually not where the crash is.** `to_fahrenheit` is fine. The mistake was passing it `"12.5"` - a string - and the string traveled two levels down before anything objected.

That gap between where a program breaks and where it went wrong is the whole reason tracebacks list several frames rather than one.

`try` and `except`

So far, an exception ends the program. `try` / `except` is how we say *if this goes wrong, here is what to do instead*.

```
try:
    <code that might fail>
except <ExceptionType>:
    <what to do if it does>
```

Python runs the `try` block. If nothing goes wrong, the `except` block is skipped entirely. If the named exception is raised, Python abandons the rest of the `try` block and runs the `except` block instead - and then carries on with the program as if nothing had happened.

In [ ]:
```
text = "forty two"

try:
    number = float(text)
```

Loading [MathJax]/extensions/Safe.js

```
    print(f"The number is {number}")
except ValueError:
    print("That is not a number.")

print("The program carries on.")
```

Change `text` to `"42"` and run it again: the `print` inside the `try` runs, the `except` block is skipped, and the last line is the same either way.

Notice the line that did **not** run when it failed. `number = float(text)` raised, so the `print` underneath it never happened - which is the point. The `try` block is a group of statements that only make sense if the first one worked.

Inside a function, this is how we turn a crash into an answer the caller can deal with.

In [ ]:
```
def to_number(text):
    """Return text as a float, or None if it cannot be read as a number."""
    try:
        return float(text)
    except ValueError:
        return None


print(to_number("42"))
print(to_number("3.14"))
print(to_number("forty two"))
```

Your turn

This function crashes on an empty list, because it divides by zero:

```
def average(values):
    return sum(values) / len(values)
```

Rewrite it so that it returns `None` instead of crashing, catching `ZeroDivisionError` by name. Check it on `[4, 9, 2]` and on `[]`.

In [ ]:
```

```

Catch the right thing

`except` can be written with no type at all, and it is almost always a mistake. Here is why.

In [ ]:
```
def to_number(text):
    try:
        return float(txt)
    except:
        return None
```

Loading [MathJax]/extensions/Safe.js

```
print(to_number("42"))
```

`"42"` is a perfectly good number, and the function said it was not.

There is a typo in there: the parameter is `text` and the body uses `txt`. That raises a `NameError` - and a bare `except:` catches **everything**, so the typo was quietly converted into "the input was bad". The function will report that every string you ever pass it is invalid, and it will never once tell you why.

Name the exception you are expecting, and anything you were not expecting is free to interrupt you - which is what you want, because being interrupted is how you find out.

In [ ]:
```
def to_number(text):
    try:
        return float(txt)
    except ValueError:
        return None


# NameError
to_number("42")
```

That is the same bug, and now it is a bug report.

Seeing the message

Writing `as` and a name gives you the exception object itself, and printing it gives you the message Python would have shown.

In [ ]:
```
for text in ["42", "forty two"]:
    try:
        print(float(text))
    except ValueError as error:
        print(f"Could not convert {text}: {error}")
```

How much should go inside the `try`?

As little as possible. Everything inside a `try` is covered by the `except` underneath it, so if two lines in there can raise the same exception, the `except` cannot tell which one did.

In [ ]:
```
price_text = "199"
quantity_text = "three"

# Too wide
    price = float(price_text)
```

Loading [MathJax]/extensions/Safe.js

```
    quantity = int(quantity_text)
    print(f"Total: {price * quantity:.2f}")
except ValueError:
    print("That is not a valid price.")
```

The price converted perfectly well. It was the quantity that failed - with a `ValueError`, the same type we were catching - and the message blamed the price. Naming the exception is not enough when two lines inside the `try` can raise it.

Put only the line your `except` is written for inside the `try`, and do the rest outside it.

In [ ]:
```
# ValueError
price_text = "199"
quantity_text = "three"

try:
    price = float(price_text)
except ValueError:
    price = None

if price is None:
    print("That is not a valid price.")
else:
    quantity = int(quantity_text)
    print(f"Total: {price * quantity:.2f}")
```

Same input, and now the error names the line that is actually wrong.

If the quantity were typed by a person too, it would deserve the same protection - and the way to give it that without writing the `try` twice is `to_number`, from a few cells up.

The proper way to check for a number

When we met `.isdigit()`, we saw that it is the wrong tool for anything that can be negative or have decimals. Now we have the right one. Here they are side by side.

In [ ]:
```
tests = ["42", "-42", "3.14", " 42 ", "", "1e5", "forty two"]

print(f"{'text':<14}{'.isdigit()':<14}float()")

for text in tests:
    try:
        converted = str(float(text))
    except ValueError:
        converted = "ValueError"

    label = '"' + text + '"'
    print(f"{label:<14}{str(text.isdigit()):<14}{converted}")
```

Loading [MathJax]/extensions/Safe.js

Every row where the two columns disagree is a row where `.isdigit()` gave the wrong answer about a perfectly good number. `-42` is a number. `3.14` is a number. `1e5` is a number - it is 100 000 written the way scientific software writes it.

The reason is not that `.isdigit()` is badly made. It answers the question it was asked, which is *is every character in this string a digit?* That is simply a different question from *can this be read as a number?*, and the second question has an authoritative answer already: hand it to `float` and see. `float` knows about minus signs, decimal points, surrounding spaces and scientific notation, because knowing about them is its whole job.

`.isdigit()` asks about the text. `float()` performs the conversion you actually wanted, and tells you if it could not. Whenever the answer you need is a number, do the second one.

`.isdigit()` is not retired. It is still exactly right for a postal code, a house number or a phone number - things that genuinely are digits and nothing else, and where `-42` should be rejected.

Your turn

Write a function `is_number(text)` that returns `True` if `text` can be read as a number and `False` if it cannot.

Check it on all five of these:

```
"0"   "-42"   "3.14.15"   "forty two"   ""
```

In [ ]:

Asking until you get an answer

Every program you have written so far gave up when the input was wrong, or re-prompted only for choices it could check with a membership test. Now we can re-prompt for a **number**, which was the case we could not do.

The cell below asks you to type something. The prompt appears at the **top of the window**, and the cell keeps running until you answer it. Type something that is not a number first, to watch the loop go round.

In [ ]:

```
def get_number(prompt):
    """Ask for a number, repeating the question until one is given."""
    while True:
        xt = input(prompt)
```

Loading [MathJax]/extensions/Safe.js

```
        try:
            return float(text)
        except ValueError:
            print("That is not a number. Please try again.")


temperature = get_number("Enter a temperature in Celsius: ")

print(f"That is {to_fahrenheit(temperature):.1f} F")
```

Three things worth noticing in eight lines:

1. The `return` is **inside** the `try`, **inside** the `while`. It only runs if the conversion succeeded, and when it does it leaves the loop and the function together - which is exactly why we do not need a flag or a `break`.

2. The loop cannot end any other way. There is no path out of `while True` except that `return`, so the function is incapable of handing back anything that is not a number. Everything downstream can stop checking.

3. And the prompt is written once. This is the `while True` shape rather than the test-in-the-header shape, for the same reason as before.

When should you *not* catch an exception?

Having learned `try` / `except`, the temptation is to put it around everything. Resist it.

A `try` / `except` says **two** things: *1) I know this can go wrong*, and *2) I know what to do instead*. The second half is the one that matters, and it is the one people forget.

Here is a function that catches an exception it has no answer for.

In [ ]:
```
def average_price(prices):
    try:
        return sum(prices) / len(prices)
    except ZeroDivisionError:
        return 0


print(average_price([199.00, 249.00, 149.00]))
print(average_price([]))
```

The exception is named, and it is the right one: an empty list divides by zero. The problem is what the function does instead. There is no average price of nothing, and `0` is a number that looks like data, goes into a report, and is believed.

Catch an exception only if you can answer **both** of these:

Loading [MathJax]/extensions/Safe.js

- **Which** exception is this? If you cannot name it, you are guessing.
- **What** will I do instead? If the answer is "carry on and hope", there is nothing to do.

If you cannot answer both, let it through. An uncaught exception is not your program failing — it is your program telling you exactly what went wrong and where. Catching it without an answer throws that away and replaces it with a wrong number.

This function had a good answer to *which* and a bad answer to *what*. There are two honest repairs: return `None`, as `average` did in the exercise above, which says *there is no value here*, or leave the `try` out and let the empty list stop the program.

The clear case *for* catching is bad input typed by a person: you know it will happen, you know which exception it produces, and asking again is genuinely better than stopping. That is what `to_number` and `get_number` do. Most other cases are not like that.

How to find a bug

Most of your programming time is spent on *debugging*. This can be especially difficult when your programs run but do the wrong thing. Follow these guidelines when checking your code:

1. Read the traceback, bottom first. The last line says what went wrong. Then walk up to the topmost frame that is your own code. If there is no traceback at all, go to step 2.

2. Print the values, not the code. Whether or not there was a traceback, put a `print` just above the suspect line showing what the variables actually *contain* - and their `type`, because nine times out of ten the answer is that something is a string when you thought it was a number.

3. Shrink the input until it still fails. A loop over 400 rows that crashes is a loop over one row that crashes. Find that row and call the function on it alone.

4. Check what a function returns before you check what it computes. A great many "wrong answer" bugs turn out to be a function that returned `None`, because it printed instead of returning, or because one branch had no `return`.

Additional resources

- Defining functions - sections 4.8 and 4.9 of the Python tutorial cover `def`, arguments and default values
- Errors and exceptions - the official account of `try`, `except`, `raise` and the built-in exception types
- Built-in exceptions - the full list, for when you meet one this notebook did not

Loading [MathJax]/extensions/Safe.js

- Debugging in VS Code - a **debugger** pauses a running script at a line you mark, so you can look at every variable at once and step forward one line at a time: the *print the values* technique above, without writing the `print` calls. We do not use it in this course, but it is worth twenty minutes of your own time once you notice yourself adding a lot of `print` calls to a script. Positron is built on the same editor, so breakpoints and the Run and Debug panel work as described here
- SKL401 - lessons 2.6 and 2.7 cover functions, default arguments and scope more briefly, and 5.6 covers reading error messages

Next week: the terminal, git and GitHub - how to move around your computer without a mouse, and how to keep every version of your work instead of `analysis_final_v3_ACTUAL.py`.

Loading [MathJax]/extensions/Safe.js